tip:所有翻译均为机翻（部分影响理解的翻译错误做了修改）

[Codeforces Round 991 (Div. 3)](#)

A. Line Breaks

difficulty:800

time limit per test:1 second

memory limit per test:256 megabytes

[2050A](#)

题解

Kostya has a text s consisting of n words made up of Latin alphabet letters. He also has two strips on which he must write the text. The first strip can hold m characters, while the second can hold as many as needed.

Kostya must choose a number x and write the first x words from s on the first strip, while all the remaining words are written on the second strip. To save space, the words are written without gaps, but each word must be entirely on one strip.

Since space on the second strip is very valuable, Kostya asks you to choose the maximum possible number x such that all words $s_1, s_2, \ldots, s_x$ fit on the first strip of length m.

Kosty有一个由n 个由拉丁字母单词组成的文本 s 。他还有两张纸条，他必须在上面写课文。第一个条带可以容纳 m 字符，而第二个条带可以根据需要容纳尽可能多的字符。

Kostya必须选择一个数字 x ，并将 s 中的第一个 x 字写在第一张纸上，而其余所有字都写在第二张纸上。为了节省空间，单词没有间隔，但每个单词必须完全在一个条带上。

由于第二个条带上的空间非常宝贵，Kostya要求您选择最大可能的数字 x ，以便所有单词 $s_1, s_2, \ldots, s_x$ 都适合长度为 m 的第一个条带。

Input

The first line contains an integer t ($1 \le t \le 1000$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \le n \le 50; 1 \le m \le 500$) — the number of words in the list and the maximum number of characters that can be on the first strip.

The next n lines contain one word s_i of lowercase Latin letters, where the length of s_i does not exceed 10.

第一行包含一个整数 t ($1 \le t \le 1000$) —测试用例的数量。

每个测试用例的第一行包含两个整数 n 和 m ($1 \le n \le 50; 1 \le m \le 500$) -列表中的单词数和第一个条带上可以出现的最大字符数。

接下来的 n 行包含一个小写拉丁字母的单词 s_i ，其中 s_i 的长度不超过 10 。

Output

For each test case, output the maximum number of words x such that the first x words have a total length of no more than m.

对于每个测试用例，输出最大单词数 x ，使第一个 x 单词的总长度不超过 m 。

Example

Input

```
5
 3 1
 a
 b
 c
 2 9
 alpha
 beta
 4 12
 hello
 world
 and
 codeforces
 3 2
 ab
 c
 d
 3 2
 abc
 ab
 a
```

Output

```
1
 2
 2
 1
 0
```

题解

按顺序加上字符的长度， 直到长度之和超过m为止。如果全加完了还没超过，输出n。

代码如下

```
t = int(input())
for _ in range(t):
    n, m = map(int, input().split())
    a = []
    for i in range(n):
        a.append(input())
    cnt = 0
    for i in range(n):
        cnt += len(a[i])
        if cnt > m:
            print(i)
            break
    else:
        print(n)
```

B. Transfusion

difficulty:1100

time limit per test:2 second

memory limit per test:256 megabytes

2050B

原题

You are given an array aof length n. In one operation, you can pick an inex i from 2 to n-1 inclusive, and do one of the following actions:

- Decrease a_{i-1} by 1, then increase a_{i1} by 1.

- Decrease a_{i1} by 1, then increase a_{i-1} by 1.

After each operation, all the values must be non-negative. Can you make all the elements equal after any number of operations?

给定一个长度为 n 的数组 a 。在一个操作中，您可以从 2 到 n-1 中选择一个索引 i ，并执行以下操作之一：

-将 a_{i-1} 减少 1 ，然后增加 a_{i1} 增加 1 。

-将 a_{i1} 减少 1 ，然后将 a_{i-1} 增加 1 。

每次操作后，所有的值都不能为负。你能在任意次数的运算后使所有元素相等吗？

Input

First line of input consists of one integer t ($1 \le t \le 10^4$) — the number of test cases. First line of each test case consists of one integer n ($3 \le n \le 2 \cdot 10^5$). Second line of each test case consists of n integers a_i ($1 \le a_i \le 10^9$). It is guaranteed that the sum of n of all test cases doesn't exceed $2 \cdot 10^5$.

第一行输入包含一个整数 t ($1 \le t \le 10^4$) —测试用例的数量。

每个测试用例的第一行由一个整数 n ($3 \le n \le 2 \cdot 10^5$)组成。

每个测试用例的第二行由 n 个整数 a_i ($1 \le a_i \le 10^9$)组成。

保证所有测试用例 n 之和不超过 $2 \cdot 10^5$ 。

Output

For each test case, print "YES" without quotation marks if it is possible to make all the elements equal after any number of operations; otherwise, print "NO" without quotation marks. You can print answers in any register: "yes", "YeS", "nO" — will also be considered correct.

对于每个测试用例，如果在任意次数的操作之后可能使所有元素相等，则打印“YES”，不带引号；否则，输出NO，不带引号。

您可以在任何寄存器中打印答案：“yes”，“yes”，“nO”-也将被认为是正确的。

Example

Input

```
8
 3
 3 2 1
 3
 1 1 3
 4
 1 2 5 4
 4
 1 6 6 1
 5
 6 2 1 4 2
 4
 1 4 2 1
 5
 3 1 2 1 3
 3
 2 4 2
```

Output

YES
NO
YES
NO
YES
NO
NO
NO

题解

思路一

题目要求把a中的所有数字变为同一个，则结果必定为这些数的平均数。操作过程中都是整数， 所以平均数不为整数的化不可能实现。

遍历数组a，尝试所有的数变为平均数，再检查每个数是否相等。

代码如下

```
t = int(input())
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))
    s = sum(a)
    if s%n !=0:
        print('NO')
        continue
    target = s//n
    f = 1
    for i in range(1, n-1):
        a[i+1] -= (target - a[i-1])
        a[i-1] = target
    for i in range(1, n):
        if a[i] != a[i-1]:
            print('NO')
            break
    else:
        print('YES')
```

思路二

不难发现所有操作均在奇数位与奇数位、偶数位与偶数位之间进行。

且所有数字变为同一个意味着奇数位所有数字变为同一个、且偶数位所有数字变为同一个、且奇数位与偶数位数字相等。

如果奇数位数字平均数不为整数、或偶数位数字平均数不为整数、或奇数位数字平均数和偶数位数字平均数不相等，则无法实现。

代码如下

```
t = int(input())
for _ in range(t):
    n = int(input())
    sum_ = [0, 0]
    cnt = [n//2 + n%2, n//2]
    a = list(map(int, input().split()))
    for i in range(n):
        sum_[i%2] += a[i]
    if sum_[0] % cnt[0] == 0 and sum_[1] % cnt[1] ==0 and sum_[0]//cnt[0] ==
sum_[1]//cnt[1]:
        print('YES')
    else:
        print('NO')
```

C. Uninteresting Number

difficulty:1200

time limit per test:2 second

memory limit per test:256 megabytes

2050C

原题

You are given a number nn with a length of no more than 10^5.

You can perform the following operation any number of times: choose one of its digits, square it, and replace the original digit with the result. The result must be a digit (that is, if you choose the digit x, then the value of x^2 must be less than 10).

Is it possible to obtain a number that is divisible by 9 through these operations?

给定一个长度不超过 10^5 的数字 n 。

您可以执行以下操作任意次：选择其中一位数字，将其平方，并将原数字替换为结果。结果必须是一个数字(也就是说，如果选择数字 x ，那么 x^2 的值必须小于 10)。

通过这些运算，是否有可能得到一个能被 9 整除的数?

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The only line of each test case contains the number n, without leading zeros. The length of the number does not exceed 10^5.

It is guaranteed that the sum of the lengths of the numbers across all test cases does not exceed 10^5.

第一行包含一个整数 t ($1 \le t \le 10^4$) —测试用例的数量。

每个测试用例的唯一一行包含数字 n，没有前导零。数字的长度不能超过 10^5 。

可以保证所有测试用例中数字长度的总和不超过 10^5 。

Output

For each test case, output "YES" if it is possible to obtain a number divisible by 9 using the described operations, and "NO" otherwise.

You can output each letter in any case (lowercase or uppercase). For example, the strings "yEs", "yes", "Yes", and "YES" will be accepted as a positive answer.

对于每个测试用例，如果使用上述操作可以得到一个能被 9 整除的数字，则输出“YES”，否则输出“NO”。

您可以输出任意大小写(小写或大写)的每个字母。例如，字符串“yEs”、“yEs”、“yEs”和“yEs”将被接受为肯定答案。

Example

Input

```
9
 123
 322
 333333333333
 9997
 5472778912773
 1234567890
 23
 33
 52254522632
```

Output

```
NO
 YES
 YES
 NO
 NO
 YES
 NO
 YES
 YES
```

题解

不难发现只有2、3是合法且更改有效的数字（0、1平方后还是其本身， 其余数的平方大于10）

一个数能整除9的充分必要条件是它的数位和能整除9。

先对每一位求和， 如果已经整除9，输出YES。

如果不整除9， 我们知道$2^2 = 4, 3^3 = 9$，即更改2的贡献为2，更改3的贡献为6。

考虑多个2和多个3的贡献，有以下式子

$$2 \times 1 \equiv 2 \mod 9, 2 \times 2 \equiv 4 \mod 9, 2 \times 3 \equiv 6 \mod 9, 2 \times 4 \equiv 8 \mod 9, 2 \times 5 \equiv 1 \mod 9, 2 \times 6 \equiv 3 \mod 9, 2 \times 7 \equiv 5 \mod 9, 2 \times 8 \equiv 7 \mod 9, 2 \times 9 \equiv 0 \mod 9, 2 \times 10 \equiv 2 \mod 9$$

以9为周期循环

$$3 \times 1 \equiv 3 \mod 9, 3 \times 2 \equiv 6 \mod 9, 3 \times 3 \equiv 0 \mod 9, 3 \times 4 \equiv 3 \mod 9$$

以3为周期循环

即在选择更改的2的数量时，有意义的选择数量为 [0,8]，选择更改的2的数量时，有意义的选择数量为 [0,2]

暴力枚举有意义的选择，判断更改后数字是否整除9即可。

代码如下

```
t = int(input())
for _ in range(t):
    s = input()
    ts = list(map(int, list(s)))
    su = sum(ts)
    count = [0, 0]
    for i in s:
        if i=='2':
            count[0] +=1
        elif i=='3':
            count[1] += 1
    if su % 9 == 0:
        print("YES")
    else:
        if count[0] == 0 and count[1] == 0:
            print('NO')
        else:
            left = su%9
            f = 0
            # 这里的范围可以缩小，比赛的时候为了保险放大了一点
            for x in range(0, min(10, count[0])+1):
                for y in range(0, min(3, count[1])+1):
                    if (left + 2*x+6*y)%9==0:
                        f = 1
                        break
                if f:
                    break
            if f:
                print("YES")
```

```
            else:
                print('NO')
```

D. Digital string maximization

difficulty:1300

time limit per test:2 second

memory limit per test:256 megabytes

[2050D](#)

原题

You are given a string s, consisting of digits from 0 to 9. In one operation, you can pick any digit in this string, except for 0 or the leftmost digit, decrease it by 1, and then swap it with the digit left to the picked.

For example, in one operation from the string 1023, you can get 1103 or 1022.

Find the lexicographically maximum string you can obtain after any number of operations.

您将得到一个字符串 s ，由从 0 到 9 的数字组成。在一个操作中，您可以选择该字符串中除了 0 或最左边的数字之外的任何数字，将其减少 1 ，然后将其与选中的左边的数字交换。

例如，对字符串 1023 进行一次操作，可以得到 1103 或 1022 。

查找在任意次数的操作之后可以获得的按字典顺序排列的最大字符串。

Input

The first line of the input consists of an integer t($1 \le t \le 10^4$) — the number of test cases.

Each test case consists of a single line consisting of a digital string s ($1 \le |s| \le 2 \cdot 10^5$), where $|s|$ denotes the length of s. The string does not contain leading zeroes.

It is guaranteed that the sum of $|s|$ of all test cases doesn't exceed $2 \cdot 10^5$.

输入的第一行包含一个整数 t($1 \le t \le 10^4$) ——测试用例的数量。

每个测试用例由一行数字字符串 s ($1 \le |s| \le 2 \cdot 10^5$)组成，其中$|s|$表示 s 的长度。

保证所有测试用例的$|s|$之和不超过 $2 \cdot 10^5$ 。

Output

For each test case, print the answer on a separate line.

对于每个测试用例，在单独的行中打印答案。

Example

Input

```
6
 19
 1709
 11555
 51476
 9876543210
 5891917899
```

Output

```
81
 6710
 33311
 55431
 9876543210
 7875567711
```

Note

In the first example, the following sequence of operations is suitable: $19 \to 81$.

In the second example, the following sequence of operations is suitable:
$1709 \to 1780 \to 6180 \to 6710$.

In the fourth example, the following sequence of operations is suitable:
$51476 \to 53176 \to 53616 \to 53651 \to 55351 \to 55431$

在第一个示例中，适用的操作顺序如下：$19 \to 81$。

在第二个示例中，适用的操作顺序如下：$1709 \to 1780 \to 6180 \to 6710$。

在第四个示例中，适用的操作顺序如下：
$51476 \to 53176 \to 53616 \to 53651 \to 55351 \to 55431$。

题解

考虑要让最后的结果经可能的大，我们应将打的数往前移动。

观察题目的最后一个样例5891917899，其移动为
$5891917899 \to 7591917899 \to 7851917899 \to 7875117899 \to$
$7875511899 \to 7875561199 \to 7875567119 \to 7875567711$

可以发现，我们在从左到右遍历的过程中总是把当前数字前移到不能前移为止，这样子能够使大的数尽可能靠前，从而使字典序最大。

代码如下

```
t = int(input())
for _ in range(t):
    s = list(map(int,list(input())))
    n = len(s)
    for i in range(1, n):
        p = i
        while (p>=1 and s[p] > s[p-1] + 1):
            s[p-1], s[p] = s[p]-1, s[p-1]
            p -= 1
    print(''.join(map(str, s)))
```

E. Three Strings

difficulty:1500

time limit per test:2.5 second

memory limit per test:256 megabytes

[2050E](#)

原题

You are given three strings: a, b, and c, consisting of lowercase Latin letters. The string c was obtained in the following way:

1. At each step, either string a or string b was randomly chosen, and the first character of the chosen string was removed from it and appended to the end of string c, until one of the strings ran out. After that, the remaining characters of the non-empty string were added to the end of c.
2. Then, a certain number of characters in string c were randomly changed.

For example, from the strings $a = \text{abra}$ and $b = \text{cada}$, without character replacements, the strings , $\text{caabdraa}, \text{abracada}, \text{acadabra}$ could be obtained.

Find the minimum number of characters that could have been changed in string c.

您将得到三个字符串： a， b 和 c， 由小写拉丁字母组成。字符串 c 通过以下方式获得：

1. 在每一步中，随机选择字符串 a 或字符串 b， 并从中删除所选字符串的第一个字符并附加到字符串 c 的末尾，直到其中一个字符串耗尽。之后，将非空字符串的剩余字符添加到 c 的末尾。
2. 然后，随机改变字符串 c 中一定数量的字符。

例如，从字符串 $a = \text{abra}$和$b = \text{cada}$ 中，不进行字符替换，可以得到字符串 $\text{caabdraa}, \text{abracada}, \text{acadabra}$。$1 \le |b| \le 10^3$

找出字符串 c 中可能被更改的最小字符数。

Input

The first line of the input contains a single integer t ($1 \le t \le 10^3$) — the number of test cases.

The first line of each test case contains one string of lowercase Latin letters a ($1 \le |a| \le 10^3$) — the first string, where $|a|$ denotes the length of string a.

The second line of each test case contains one string of lowercase Latin letters b ($1 \le |b| \le 10^3$) — the second string, where $|b|$ denotes the length of string b.

The third line of each test case contains one string of lowercase Latin letters c ($|c| = |a||b|$) — the third string.

It is guaranteed that the sum of $|a|$ across all test cases does not exceed $2 \cdot 10^3$. Also, the sum of $|b|$ across all test cases does not exceed $2 \cdot 10^3$.

输入的第一行包含一个整数 t ($1 \le t \le 10^3$)——测试用例的数量。

每个测试用例的第一行包含一个小写拉丁字母字符串 a ($1 \le |a| \le 10^3$)——第一个字符串，其中$|a|$表示字符串 a 的长度。

每个测试用例的第二行包含一个小写拉丁字母字符串 b ($1 \le |b| \le 10^3$)第二个字符串，其中$|b|$表示字符串 b 的长度。

每个测试用例的第三行包含一个小写拉丁字母字符串 c ($|c| = |a||b|$)——第三个字符串。

可以保证所有测试用例的$|a|$之和不超过 $2 \cdot 10^3$。同样，所有测试用例的$|b|$之和不超过 $2 \cdot 10^3$。

Output

For each test case, output a single integer — the minimum number of characters that could have been changed in string c.

对于每个测试用例，输出一个整数—字符串 c 中可以被更改的最小字符数。

Example

Input

```
7
a
b
cb
ab
cd
acbd
ab
```

ba
aabb
xxx
yyy
xyxyxy
a
bcd
decf
codes
horse
codeforces
egg
annie
egaegaeg

Output

1
0
2
0
3
2
3

题解

可以用动态规划求解

令dp[i][j]为使用了a的前i个字符和b的前j个字符时c更改的字符数量的最小值。

有

![dp[i][0] = dp[i-1][0] + (a[i-1]!=c[i-1])](https://latex.csdn.net/eq?dp%5Bi%5D%5B0%5D%20%3D%20dp%5Bi-1%5D%5B0%5D%20+%20%28a%5Bi-1%5D%21%3Dc%5Bi-1%5D%5D%29)

$$dp[0][j] = dp[0][j-1](b[j-1]! = c[j-1])$$

$$dp[i][j] = min(dp[i-1][j](a[i-1]! = c[i-1j]), dp[i][j-1](b[j-1]! = c[j-1i]))$$

i, j 满足$(1 \leqslant i \leqslant |a|, 1 \leqslant j \leqslant |b|)$

代码如下

```
t = int(input())
for _ in range(t):
    a = input()
    b = input()
    c = input()
    na = len(a)
    nb = len(b)
    dp = [[10**9 for i in range(nb+1)] for j in range(na+1)]
```

```
    dp[0][0] = 0
    for i in range(1, na+1):
        dp[i][0] = dp[i-1][0] + (a[i-1]!=c[i-1])
    for j in range(1,nb+1):
        dp[0][j] = dp[0][j-1] + (b[j-1]!=c[j-1])
    for i in range(1, na+1):
        for j in range(1, nb+1):
            dp[i][j] = min(dp[i-1][j] + (a[i-1]!=c[i-1+j]), dp[i][j-1] + (b[j-
1]!=c[j-1+i]))
    print(dp[na][nb])
```

F. Maximum modulo equality

difficulty:1700

time limit per test:2.5 second

memory limit per test:256 megabytes

2050F

原题

You are given an array a of length n and q queries l, r.

For each query, find the maximum possible m, such that all elements $a_l, a_{l1}, \ldots, a_r$ are equal modulo m. In other words, $a_l \bmod m = a_{l1} \bmod m = \cdots = a_r \bmod m$, where $a \bmod b$ — is the remainder of division a by b. In particular, when m can be infinite, print 0.

给定一个长度为 n 的数组 a ， 查询为 l 、 r 的 q 。

对于每个查询，找出最大可能的 m，使得所有元素 $a_l, a_{l1}, \ldots, a_r$ 模m相等。换句话说，就是 $a_l \bmod m = a_{l1} \bmod m = \cdots = a_r \bmod m$ ，其中 $a \bmod b$ -是 a 除以 b 的余数。特别是当 m 可以是无穷大时，打印 0 。

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains two integers n, q ($1 \le n, q \le 2 \cdot 10^5$) — the length of the array and the number of queries.

The second line of each test case contains n integers a_i ($1 \le a_i \le 10^9$) — the elements of the array.

In the following q lines of each test case, two integers l, r are provided ($1 \le l \le r \le n$) — the range of the query.

It is guaranteed that the sum of n across all test cases does not exceed $2 \cdot 10^5$, and the sum of q does not exceed $2 \cdot 10^5$.

第一行包含单个整数 t ($1 \le t \le 10^4$)—测试用例的数量。

每个测试用例的第一行包含两个整数 n, q ($1 \le n, q \le 2 \cdot 10^5$)——数组的长度和查询的数量。

每个测试用例的第二行包含 n 个整数 a_i ($1 \le a_i \le 10^9$) ——数组的元素。

在接下来的每个测试用例的 q 行中，提供了两个整数 l, r ($1 \le l \le r \le n$)-查询的范围。

保证所有测试用例 n 的和不超过 $2 \cdot 10^5$，q 的和不超过 $2 \cdot 10^5$。

Output

For each query, output the maximum value m described in the statement.

对于每个查询，输出语句中描述的最大值 m 。

Example

Input

```
3
 5 5
 5 14 2 6 3
 4 5
 1 4
 2 4
 3 5
 1 1
 1 1
 7
 1 1
 3 2
 1 7 8
 2 3
 1 2
```

Output

```
 3 1 4 1 0
 0
 1 6
```

题解

$a_l \bmod m = a_{l1} \bmod m = \cdots = a_r \bmod m$, 则
$(a_r - a_{r-1}) \mod m = (a_{r-1} - a_{r-2}) \mod m = \cdots = (a_{l1} - a_l) \mod m = 0$

即m为$\gcd(a_r - a_{r-1}, a_{r-1} - a_{r-2}, \ldots, a_{l1} - a_l)$,当$l = r$时m为0

因为没有修改操作， 用st表处理会更加高效

st表相关知识详见[ST 表 - OI Wiki]

代码如下

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

const ll MAXN = 200002;
ll a[MAXN], b[MAXN], LOG2[MAXN], dp[MAXN][22];

void solve(){
    a[0] = 0;
    ll n = read(), q =read();
    for(ll i=1; i<=n;i++){
        a[i] = read();
    }
    for(ll i=1; i<=n-1;i++){
        b[i] = abs(a[i+1]-a[i]);
        dp[i][0] = b[i];
    }
    ll p = LOG2[n-1];
    for(ll k=1;k<=p;k++)
    {
        for(ll s=1;s+(1<<k)<=n;s++)
        {
            dp[s][k] = __gcd(dp[s][k-1], dp[s+(1<<(k-1))][k-1]);
        }
    }
    ll l, r, k;
    while(q--){
        l = read(), r = read();
        if (l==r){
            pt(0), putchar(' ');
            continue;
        }
        r--;
        k = LOG2[r-l+1];
        pt(__gcd(dp[l][k], dp[r-(1<<k)+1][k])), putchar(' ');
    }
    puts("");
```

```
}

int main(){
    LOG2[0] = -1;
    for(ll i=1;i<=MAXN;i++){
        LOG2[i] = LOG2[i>>1] + 1;
    }
    ll t = read();
    while(t--){
        solve();
    }
}
```

G.Tree Destruction

difficulty:1900

time limit per test:2 second

memory limit per test:256 megabytes

2050G

原题

Given a tree* with n vertices. You can choose two vertices a and b once and remove all vertices on the path from a to b, including the vertices themselves. If you choose a=b, only one vertex will be removed.

Your task is to find the maximum number of connected components† that can be formed after removing the path from the tree.

* tree is a connected graph without cycles.

†A connected component is a set of vertices such that there is a path along the edges from any vertex to any other vertex in the set (and it is not possible to reach vertices not belonging to this set)

给定一棵有 n 个顶点的树 * 。您可以选择两个顶点 a 和 b ，然后删除从 a 到 b 的路径上的所有顶点，包括顶点本身。如果您选择 a=b ，则只会移除一个顶点。

你的任务是找出从树中移除路径后所能形成的最大连通组件数 † 。

* 树是一个没有循环的连通图。

† 一个连通组件是一个顶点集合：从任意顶点到集合中的任意其他顶点都有一条沿边的路径（并且不可能到达不属于这个集合的顶点）。

Input

The first line of the input contains one integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains one integer n ($2 \le n \le 2 \cdot 10^5$) — the size of the tree.

The next n-1 lines contain two integers u and v($1 \le u, v \le n, u \ne v$) — the vertices connected by an edge. It is guaranteed that the edges form a tree.

It is guaranteed that the sum of n across all test cases does not exceed $2 \cdot 10^5$.

输入的第一行包含一个整数 t ($1 \le t \le 10^4$) - 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($2 \le n \le 2 \cdot 10^5$) - 树的大小。

接下来的 n-1 行包含两个整数 u 和 v ($1 \le u, v \le n, u \ne v$)--由一条边连接的顶点。可以保证这些边构成一棵树。

保证所有测试用例中 n 的总和不超过 $2 \cdot 10^5$。

Output

For each test case, output one integer — the maximum number of connected components that can be achieved using the described operation.

对于每个测试用例，输出一个整数 - 使用所述操作可实现的最大连接组件数。

Example

Input

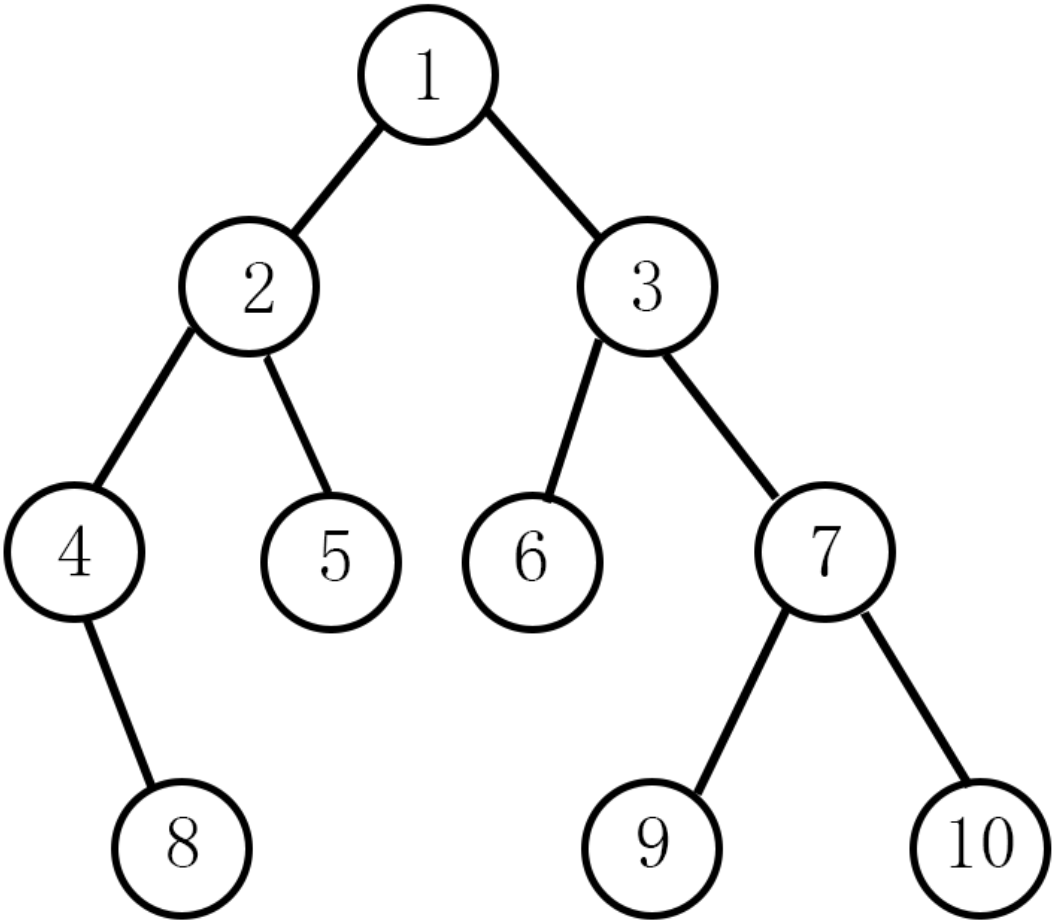
```
6
 2
 1 2
 5
 1 2
 2 3
 3 4
 3 5
 4
 1 2
 2 3
 3 4
 5
 2 1
 3 1
 4 1
 5 4
 6
 2 1
 3 1
 4 1
 5 3
 6 3
```

```
6
2 1
3 2
4 2
5 3
6 4
```

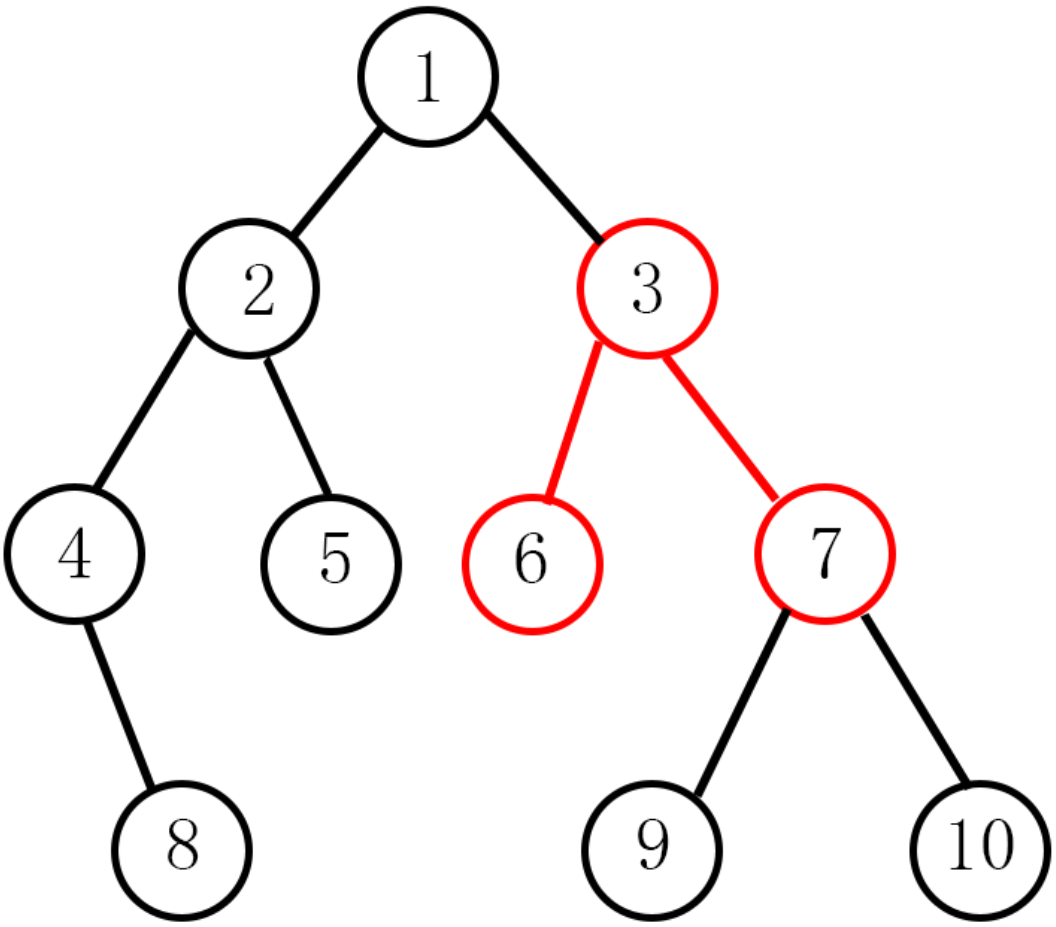
Output

```
1
3
2
3
4
3
```

题解



编辑



编辑

若一条路径上有k条边， 路径点上的的度之和为s，则树会分裂成$s-2\times k$部分。

不妨观察上图中的树，若选择$6\to 3\to 7$的路径，会分裂为$(132)-2\times 2=2$

不难发现，树能裂成几部分取决于路径和路径外的点有多少条边链接。

在计算路径上的点的度之和时，路径上每条边都被计算了两次。所以剩余的$s-2\times k$条边即为连通组件的个数。

对于每个节点，我们计算由根节点出发的路径的最大答案，以及经过根节点的路径的最大答案。

两个值均可以有子节点推到父节点。

对于第一个值$dp1_u=max(d(u),d(u)dp1_v-2),v\subseteq graph[u]$（减2是因为链接上u节点后路径上多了一条边）

对于第二个值$dp2_u=d(u)max1(dp1_v)max2(dp1_v)-4,v\subseteq graph[u]$（减4是因为链接上u节点后路径上多了两条边）

其中d(u)代表u的度，即有多少条边和u相连、max1代表最大值、max2代表次大值。

答案为所有节点的两个值的最大值

代码如下

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef __int128 INT;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll inf =  0x3f3f3f3f;
const ll INF = INT_MAX;
const ll MOD = 1000000007;
const ll mod = 998244353;
const ll maxn = 200005;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}


void solve(){
    ll n = read(), u, v;
    vector<PLL> dp(n+1);
    vector<vector<ll>> graph(n+1);
    for(ll i=0;i<n-1;i++){
        u = read();
        v = read();
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    function<void(ll , ll)> dfs = [&](ll u, ll p){
        dp[u].first = graph[u].size();//从父节点出发的一条路径的最大答案
        ll m1 = -1, m2 = -1;
        for(auto v:graph[u]){
            if (v == p) continue;
            dfs(v, u);
            dp[u].first = max(dp[u].first, dp[v].first + (ll)graph[u].size() -
2ll * 1ll);
            if (dp[v].first > m1){
                m2 = m1;
                m1 = dp[v].first;
            }
            else if (dp[v].first > m2){
                m2 = dp[v].first;
            }
        }
        if (m2 == -1){//只有一个子节点
            dp[u].second = dp[u].first;
        }
```

```
        else{
            dp[u].second = m1 + m2 + graph[u].size() - 2 * 2;
        }
    };
    dfs(1ll, -1ll);

    ll ans = 1;
    for(ll i=1;i<=n;i++){
        ans = max(ans, max(dp[i].first, dp[i].second));
    }
    print(ans);
}

int main(){
    ll t = read();
    while (t--){
        solve();
    }
}
```